Trabajo Práctico 8 — Resolución

Programación Lógica Rebatible (DeLP)

Inteligencia Artificial — UNS — 2026

Las fuentes van en notas al pie. **Cátedra**: diapositivas de A. J. García (2026). **AIMA**: Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 3.ª ed. (2010). **PMG**: Poole, Mackworth & Goebel, *Computational Intelligence: A Logical Approach* (1998).

Índice

Ejercicio 1: Conceptos básicos **3**

a) . . . 3
b) . . . 3
c) . . . 4
d) . . . 6

Ejercicio 2 **8**

a) . . . 8
b) . . . 9
c) . . . 10
d) . . . 10

Ejercicio 3: Argumentos **12**

a) . . . 12
b) . . . 13
c) . . . 14
d) . . . 15

Ejercicio 4: Ataques y derrotas **16**

a) . . . 16
b) . . . 17
c) . . . 17

Ejercicio 5 **19**

a) . . . 20
b) . . . 20

Ejercicio 6: Líneas de argumentación **22**

a) . . . 22
b) . . . 23

Ejercicio 7 **25**

Ejercicio 8: Análisis dialéctico y respuesta a consultas **27**

a) 27

b) 27

c) 28

d) 29

e) 29

f) 30

g) 32

h) 33

Ejercicio 1: Conceptos básicos

a)

Consigna

¿Qué es un programa lógico rebatible?

Un **programa lógico rebatible** (PLR) es un par (Π, Δ) (o el conjunto $\Pi \cup \Delta$) formado por:[1]

- Π: un conjunto de **hechos** y **reglas estrictas**.
  - *Hecho*: un literal fijo (sin variables).
  - *Regla estricta* $L_0 \leftarrow L_1, \ldots, L_n$: información **segura**.
- Δ: un conjunto de **reglas rebatibles** $L_0 \prec L_1, \ldots, L_n$: información **tentativa**.

Un **literal** es un átomo $p(T_1, \ldots, T_n)$ o un átomo negado $\sim p(T_1, \ldots, T_n)$, donde "~" es la **negación fuerte**.

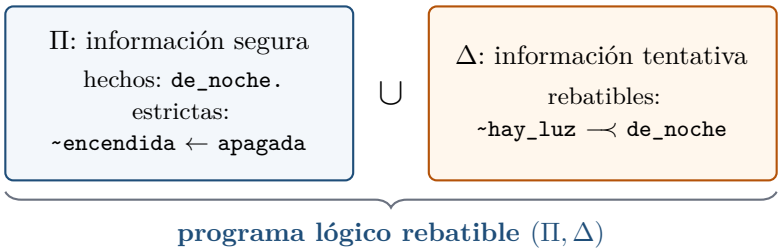


Figura 1: Componentes de un PLR (con reglas del inciso d como ejemplo).

EXTRAS

- DeLP provee un **lenguaje de representación de conocimiento** que permite representar información **tentativa** y **potencialmente en conflicto**. Ante una consulta, su mecanismo de inferencia construye y evalúa **todos** los argumentos relacionados con ella (a favor y en contra).[2]
- **Variables**: las mayúsculas en los literales denotan variables; una regla con variables es un **esquema de reglas**. Los hechos, en cambio, son los encargados de **propagar las instancias** en las derivaciones de las conclusiones.[3]
- La base es la tesis doctoral de A. J. García, *Programación Lógica Rebatible* (UNS, 2000), cap. 2, y el artículo de García y Simari, *Defeasible logic programming: An argumentative approach* (TPLP, 2004).[4]

b)

Consigna

¿Cuál es la utilidad de contar con dos tipos de reglas: estrictas y rebatibles? ¿Qué tipo de información permite modelar cada una de estas reglas? ¿Cómo se distinguen estas reglas en el lenguaje?

[1] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)» y «Hechos (facts)».
[2] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».
[3] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)» y «Hechos (facts)».
[4] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».

Qué modela cada una.[5]

- **Estricta**: información **segura** (no rebatible). La conexión entre el antecedente y la conclusión **se cumple siempre**: no puede haber excepciones. Ej.: `~abierto` ← `cerrado`.
- **Rebatible**: información **tentativa**. La conexión entre el antecedente (las razones) y la conclusión se cumple en muchos casos, pero **puede haber excepciones**. Ej.: `~cruzar` —≺ `semaforo(rojo)`.

Utilidad de tener las dos.

- Con las rebatibles se pueden expresar **reglas generales** que se cumplen en la mayoría de los casos y, además, reglas que dicen **en qué circunstancias hay excepciones**; también **posturas u opiniones diferentes** que pueden venir de distintas fuentes.[6]
- Con las estrictas se representa lo que no admite excepciones. Si hay excepciones, no es estricta y debe representarse como rebatible; la decisión depende del **dominio de aplicación** del SBC y es responsabilidad de quien codifica el conocimiento.[7]

Cómo se distinguen. Sintácticamente, **solo por la flecha** (aunque la diferencia semántica es enorme):[8]

Regla	Notación	Información que modela	Ejemplo
Estricta	$L_0 \leftarrow L_1, \ldots, L_n$	segura: se cumple siempre, sin excepciones	`~abierto` ← `cerrado`
Rebatible	$L_0 \prec\!\!- L_1, \ldots, L_n$	tentativa: se cumple en muchos casos, puede tener excepciones	`~cruzar` —≺ `semaforo(rojo)`

EXTRAS

- **Lectura de una regla rebatible** `conclusión` —≺ `antecedente`: "si hay razones para creer en el antecedente, entonces hay una razón (rebatible) para creer en la conclusión".[9]
- **Cadena de excepciones**: cada regla puede ser excepción de la anterior. Según los hechos que perciba el agente, pueden derivar conclusiones las tres, solo dos, solo una o ninguna:[10]

`~cruzar` —≺ `semaforo(rojo).`	regla general
`cruzar` —≺ `semaforo(rojo), emergencia.`	excepción a la primera
`~cruzar` —≺ `semaforo(rojo), emergencia, hay_peligro.`	excepción a la segunda

c)

Consigna

¿Qué ventajas tiene usar DeLP como mecanismo de inferencia y/o como herramienta de representación de conocimiento frente a otros enfoques, como el de la lógica clásica o frente a Prolog?

Ventajas de DeLP.[11]

[5] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)» y «Reglas estrictas (strict rules)».
[6] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».
[7] Cátedra, Episodio X, parte 2, «Reglas estrictas (strict rules)».
[8] Cátedra, Episodio X, parte 2, «Reglas estrictas (strict rules)».
[9] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)».
[10] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)».
[11] Cátedra, Episodio X, parte 2, «Defeasible Logic Programming». Mismo resumen en Episodio X (tercera parte), «Comentarios finales».

- *Representación*: permite representar información **tentativa** y información **potencialmente en conflicto**.
- *Inferencia*:
  - las conclusiones están **sustentadas con argumentos**;
  - las respuestas se basan en un **análisis exhaustivo** de todos los argumentos relacionados (a favor y en contra);
  - el criterio de preferencia entre argumentos se puede **adaptar al dominio** de aplicación;
  - se puede dar una **explicación detallada** de las respuestas.

Frente a Prolog.

- Prolog estándar **no tiene negación fuerte**, y tampoco un mecanismo para **manejar información contradictoria**: si hay derivaciones para `comprar_acciones(acme)` y para `no_comprar_acciones(acme)`, responde `true` a las dos.[12]
- Solo tiene **negación por falla** (`\+`), que no puede ir en la cabeza de una regla ni en un hecho.[13]
- DeLP agrega la negación fuerte (~) y las reglas rebatibles, y ante conclusiones contradictorias **decide cuál prevalece** comparando los argumentos a favor y en contra.[14]

Frente a la lógica clásica.

- La lógica clásica es **monotónica**:[15] "*the set of entailed sentences can only increase as information is added to the knowledge base*". Lo que se concluyó no se retracta al aparecer una excepción.
- Para no inferir conclusiones contradictorias con inferencia monotónica (como Modus Ponens) hay que **mantener la base libre de contradicciones**. La otra opción es **admitir contradicciones potenciales** en la base y tener una inferencia (no monotónica) que **decida** ante una contradicción.[16] DeLP sigue esta segunda opción.

	Lógica clásica	**Prolog**	**DeLP**
Negación	—	solo por falla (`\+`)	fuerte (~)
Información tentativa	no (monotónica)	no	sí (reglas rebatibles)
Contradicciones	base libre de contradicciones	concluye las dos	decide cuál prevalece

EXTRAS

- **No monotonía en AIMA**: razonar por defecto, sacando conclusiones "*in the absence of any reason to doubt it*" y retractándolas si aparece nueva evidencia, es habitual en el sentido común. Ese razonamiento "*is said to exhibit nonmonotonicity, because the set of beliefs does not grow monotonically over time as new evidence arrives*". Ejemplos de lógicas no monotónicas: *circumscription* y *default logic*.[17]
- **Un paso intermedio sobre Prolog**: agregarle el operador ~ y una inferencia "cauta" que responda `si` (deriva Q pero no su complemento), `no` (deriva el complemento pero no Q),

[12] Cátedra, Episodio VIII, «Extender el mecanismo de inferencia».
[13] Cátedra, Episodio VIII, «Negación por falla en Prolog».
[14] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».
[15] AIMA, §7.5, p. 251.
[16] Cátedra, Episodio IX (valores de certeza), «Bases de creencias inconsistentes: dos grandes opciones».
[17] AIMA, §12.6.1, p. 458.

`indeciso` (deriva ambos) o `desconozco` (ninguno). Ante una contradicción no decide: se abstiene.[18]

d)

Consigna

Escriba reglas estrictas o rebatibles, según considere más adecuado, para representar el siguiente conocimiento:

I. Si la linterna está apagada, entonces no está encendida.
II. Si la linterna está encendida, entonces no está apagada.
III. Si es de noche, generalmente no hay luz.
IV. Si es de noche y tengo una linterna, entonces puedo iluminar.
V. Si la linterna no tiene batería, entonces no ilumina.
VI. Si es de noche y hay luna llena, entonces hay luz.

Supuesto

"Puedo iluminar" (iv) y "no ilumina" (v) se representan con el mismo literal `ilumina` (y su complemento), y "hay luz" (iii y vi) con `hay_luz`; si no, las reglas no entrarían en conflicto.

Criterio: si la regla puede tener excepciones, es rebatible; si se cumple siempre, estricta.[19]

	Regla	Tipo	Por qué
i	`~encendida` ← `apagada`	estricta	por definición: apagada y encendida son opuestos, sin excepciones
ii	`~apagada` ← `encendida`	estricta	ídem
iii	`~hay_luz` —≺ `de_noche`	rebatible	"generalmente": tiene excepciones (vi)
iv	`ilumina` —≺ `de_noche, tengo_linterna`	rebatible	tiene excepciones: v
v	`~ilumina` ← `~tiene_bateria`	estricta	sin batería, la linterna no ilumina nunca
vi	`hay_luz` —≺ `de_noche, luna_llena`	rebatible	es una excepción a iii, y a su vez puede tener excepciones (p. ej. noche nublada)

- iv no dice "generalmente", pero v es una excepción: tener linterna de noche no basta si no tiene batería. Si iv fuera estricta, con `de_noche`, `tengo_linterna` y `~tiene_bateria` en Π se derivarían `ilumina` y `~ilumina` solo con información segura.
- iii y vi entran en conflicto (`~hay_luz` vs. `hay_luz`) en una noche de luna llena: las dos son razones tentativas y será la comparación de argumentos la que decida.

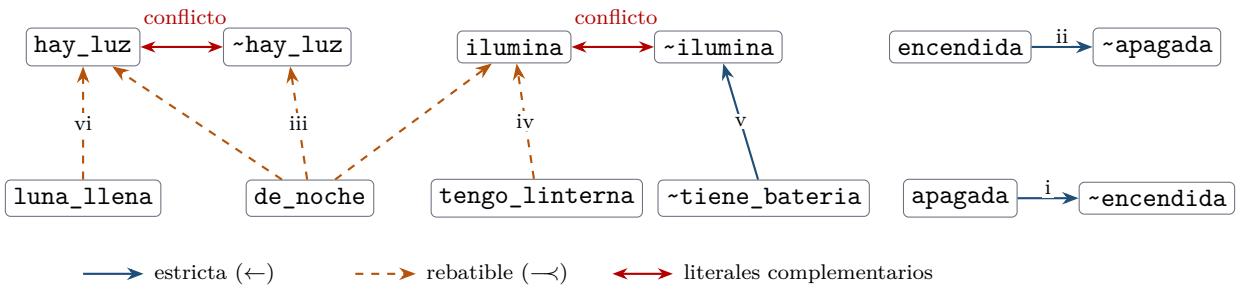


Figura 2: Las reglas del inciso d: de los antecedentes a la conclusión, y los conflictos entre conclusiones.

[18] Cátedra, Episodio VIII, «Extendiendo el mecanismo de inferencia».
[19] Cátedra, Episodio X, parte 2, «Reglas estrictas (strict rules)».

EXTRAS

- Con v estricta, si `~tiene_bateria` es un hecho, el conjunto {iv} junto con Π es contradictorio (deriva `ilumina` y `~ilumina`), así que no es un argumento para `ilumina`: no cumple la condición de consistencia.[20]

[20] Cátedra, Episodio X, parte 2, «Argumento».

Ejercicio 2

Consigna

Considere el siguiente programa lógico rebatible $\mathcal{P}_2 = (\Pi_2, \Delta_2)$:

$$\Pi_2 = \left\{ \begin{array}{l} asustada(tina) \\ gallina(tina) \\ gallina(peluche) \\ \sim vuela(peluche) \\ ave(X) \leftarrow gallina(X) \end{array} \right\} \qquad \Delta_2 = \left\{ \begin{array}{l} vuela(X) \prec ave(X) \\ \sim vuela(X) \prec gallina(X) \\ vuela(X) \prec gallina(X), asustada(X) \\ anida_en_ramas_altas(X) \prec vuela(X) \end{array} \right\}$$

En lo que sigue, las reglas se nombran así:

E1:	`ave(X)` $\leftarrow$ `gallina(X)`	R2:	`~vuela(X)` $\prec$ `gallina(X)`
R1:	`vuela(X)` $\prec$ `ave(X)`	R3:	`vuela(X)` $\prec$ `gallina(X), asustada(X)`
		R4:	`anida_en_ramas_altas(X)` $\prec$ `vuela(X)`

a)

Consigna

¿A qué se denomina derivación rebatible? Obtenga todas las derivaciones para los siguientes literales: vuela(tina), ~vuela(tina), asustada(tina), vuela(peluche) y ~vuela(peluche).

Derivación rebatible. Una **derivación rebatible** para un literal L a partir de un PLR (Π, Δ) es una secuencia finita de literales fijos $L_1, L_2, \ldots, L_n = L$ tal que, para cada L_i:[21]

- L_i es un **hecho** de Π, o
- existe en (Π, Δ) una **instancia de una regla** (estricta o rebatible) con cabeza L_i y cuerpo $B_1, \ldots, B_k$, donde todo literal del cuerpo ya está en $L_1, \ldots, L_{i-1}$.

Si solo usa hechos y reglas estrictas de Π, es una **derivación estricta**.

Qué se cuenta como "todas"

Cambiar el orden de literales independientes (p. ej. `asustada(tina)` antes o después de `gallina(tina)`) o agregar literales que no se usan da otras secuencias que también cumplen la definición. Se listan las derivaciones distintas en cuanto a **qué hechos y reglas usan**.

[21] Cátedra, Episodio X, parte 2, «Derivaciones rebatibles (defeasible derivation)».

Literal		Secuencia	Justificación de cada L_i	Tipo
`vuela(tina)`	D1	`gallina(tina)`, `ave(tina)`, `vuela(tina)`	hecho E1 instanciada R1 instanciada	rebatible
	D2	`gallina(tina)`, `asustada(tina)`, `vuela(tina)`	hecho hecho R3 instanciada	rebatible
`~vuela(tina)`	D3	`gallina(tina)`, `~vuela(tina)`	hecho R2 instanciada	rebatible
`asustada(tina)`	D4	`asustada(tina)`	hecho	estricta
`vuela(peluche)`	D5	`gallina(peluche)`, `ave(peluche)`, `vuela(peluche)`	hecho E1 instanciada R1 instanciada	rebatible
`~vuela(peluche)`	D6	`~vuela(peluche)`	hecho	estricta
	D7	`gallina(peluche)`, `~vuela(peluche)`	hecho R2 instanciada	rebatible

- R3 no sirve para `vuela(peluche)`: no hay forma de derivar `asustada(peluche)`.
- R4 no aparece porque ninguno de los literales pedidos es `anida_en_ramas_altas(...)`.

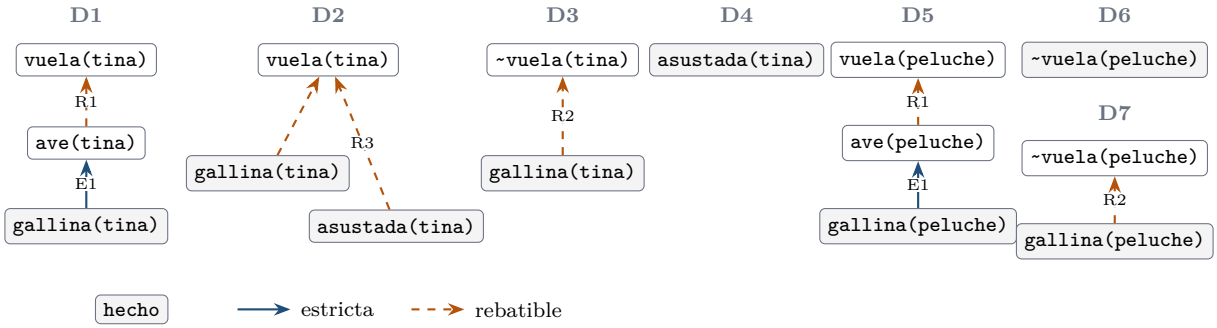


Figura 3: Las derivaciones D1–D7 como árboles: cada literal se obtiene de los de abajo con la regla indicada.

EXTRAS

- D1 y D5 muestran que una derivación puede usar **los dos tipos de reglas** (E1 estricta y R1 rebatible).[22]
- Los literales complementarios respecto de "~" (como `vuela(tina)` y `~vuela(tina)`) también se llaman **literales contradictorios** o **en conflicto**, porque representan dos conclusiones contradictorias.[23]

b)

Consigna

¿Cuándo se dice que un conjunto de reglas es contradictorio? Brinde un ejemplo.

Un conjunto S de hechos, reglas estrictas y rebatibles es **contradictorio** si a partir de S se pueden obtener derivaciones rebatibles tanto para un literal p como para su complemento $\sim p$.[24]

[22] Cátedra, Episodio X, parte 2, «Derivaciones para literales complementarios».
[23] Cátedra, Episodio X, parte 2, «Derivaciones para literales complementarios».
[24] Cátedra, Episodio X, parte 2, «Conjunto de reglas contradictorio».

Ejemplo (de la cátedra):[25]

$$\Pi_c = \left\{ \begin{array}{l} \texttt{en_supermercado.} \\ \texttt{en_casa.} \\ \texttt{\sim en_casa} \leftarrow \texttt{en_supermercado.} \end{array} \right\}$$

- `en_casa` se deriva porque es un hecho.
- `~en_casa` se deriva con la secuencia `en_supermercado`, `~en_casa`.

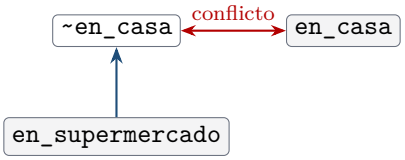


Figura 4: Π_c deriva dos literales complementarios: es contradictorio.

c)

Consigna

¿Cuál es la utilidad de que un programa lógico rebatible sea contradictorio? ¿Tiene sentido que el conjunto Π sea contradictorio?

Utilidad de un PLR contradictorio. Es lo que permite representar:[26]

- **información tentativa**: reglas generales y reglas para sus **excepciones**. En $\mathcal{P}_2$, R1 (las aves vuelan) tiene la excepción R2 (las gallinas no vuelan), que a su vez tiene la excepción R3 (las gallinas asustadas vuelan);
- **información potencialmente en conflicto**: distintas posturas u opiniones, que pueden venir de distintas fuentes.

Las contradicciones no se evitan: las resuelve el mecanismo de inferencia, analizando los argumentos a favor y en contra.

¿Π contradictorio? No. En todo PLR (Π, Δ) **se asume que** Π **no es contradictorio**: los hechos y las reglas estrictas (información segura) no pueden inferir conclusiones contradictorias.[27]

- Un conjunto contradictorio como Π_c del inciso b **no es válido** en DeLP como Π.
- Además, un argumento A exige que $\Pi \cup A$ no sea contradictorio. Con Π contradictorio, $\Pi \cup A$ lo sería para todo A, y no habría ningún argumento.[28]

d)

Consigna

Justifique por qué $(\Pi_2 \cup \Delta_2)$ es contradictorio, y por qué Π_2 no es contradictorio.

- $\Pi_2 \cup \Delta_2$ **es contradictorio**: deriva dos pares de literales complementarios (inciso a):
  - `vuela(tina)` (D1 o D2) y `~vuela(tina)` (D3);

[25] Cátedra, Episodio X, parte 2, «Conjunto de reglas contradictorio».
[26] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».
[27] Cátedra, Episodio X, parte 2, «Conjunto de reglas contradictorio».
[28] Cátedra, Episodio X, parte 2, «Argumento».

- `vuela(peluche)` (D5) y `~vuela(peluche)` (D6 o D7).

- Π_2 **no es contradictorio**: solo con hechos y E1 se derivan los cuatro hechos más `ave(tina)` y `ave(peluche)` (E1 aplicada a cada gallina). Entre estos seis literales no hay ningún par p, $\sim p$: el único literal negado, `~vuela(peluche)`, no tiene su complemento `vuela(peluche)`, que solo se obtiene con R1.

Literal	**Desde** Π_2	**Desde** $\Pi_2 \cup \Delta_2$
`asustada(tina)`, `gallina(tina)`, `gallina(peluche)`	sí (hechos)	sí
`ave(tina)`, `ave(peluche)`	sí (E1)	sí
`~vuela(peluche)`	sí (hecho)	sí
`vuela(peluche)`	no	**sí** (D5) ⇒ conflicto
`vuela(tina)`	no	**sí** (D1, D2)
`~vuela(tina)`	no	**sí** (D3) ⇒ conflicto con la anterior

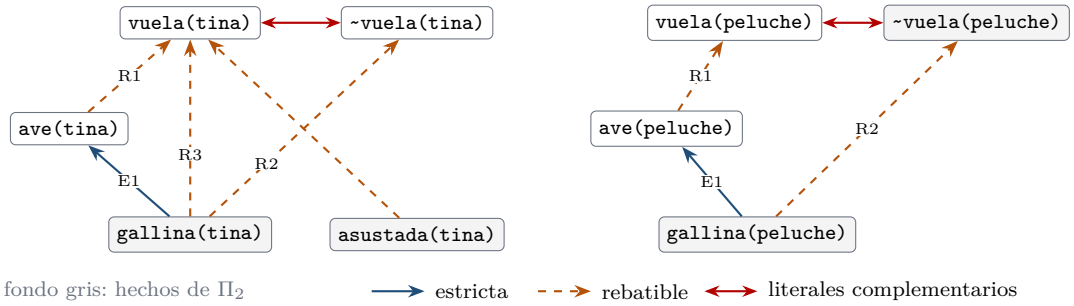


Figura 5: Lo que se deriva de $\Pi_2 \cup \Delta_2$ para `vuela`. Los dos conflictos aparecen recién al usar reglas rebatibles (líneas punteadas).

Ejercicio 3: Argumentos

a)

Consigna

¿Cómo se define formalmente la noción de *argumento* en DeLP? Explique cada una de las condiciones exigidas en la definición de argumento.

Un **argumento** para una conclusión h a partir de un PLR (Π, Δ) es un par $\langle A, h\rangle$, donde h es un literal fijo y A es un conjunto de **reglas rebatibles instanciadas** de Δ que cumple:[29]

1. **Derivación**: existe una derivación rebatible para h a partir de $A \cup \Pi$.
2. **Consistencia**: $A \cup \Pi$ no es contradictorio.
3. **Minimalidad**: no existe un subconjunto propio de A que cumpla (1) y (2).

Qué pide cada condición. Con los ejemplos de la cátedra, a partir de:[30]

$$\Pi_n = \left\{ \begin{array}{l} \texttt{~en_casa} \leftarrow \texttt{en_supermercado.} \\ \texttt{en_supermercado.} \\ \texttt{de_noche.} \\ \texttt{festejo.} \end{array} \right\} \qquad \Delta_n = \left\{ \begin{array}{l} \texttt{en_casa} \prec \texttt{de_noche.} \\ \texttt{preparo_cena} \prec \texttt{de_noche.} \\ \texttt{~preparo_cena} \prec \texttt{festejo.} \\ \texttt{acuesto_tarde} \prec \texttt{festejo.} \\ \texttt{en_oficina} \prec \texttt{trabajando.} \end{array} \right\}$$

- **(1) Derivación**: las reglas de A, junto con los hechos y reglas estrictas, tienen que permitir **llegar a** h. $Q = \{\texttt{en_oficina} \prec \texttt{trabajando}\}$ no es argumento para `en_oficina`: `trabajando` no se deriva, así que $Q \cup \Pi_n$ no deriva `en_oficina`.
- **(2) Consistencia**: las reglas de A **no pueden contradecir** lo que se sabe con seguridad ni contradecirse entre sí. $R = \{\texttt{en_casa} \prec \texttt{de_noche}\}$ no es argumento para `en_casa`: $R \cup \Pi_n$ deriva `en_casa` y `~en_casa`.
- **(3) Minimalidad**: que **no haya reglas de más**; todas son necesarias para la derivación. $S = \{\texttt{~preparo_cena} \prec \texttt{festejo}, \texttt{acuesto_tarde} \prec \texttt{festejo}\}$ no es argumento para `~preparo_cena`, porque $B = \{\texttt{~preparo_cena} \prec \texttt{festejo}\} \subset S$ ya cumple (1) y (2). $\langle B, \texttt{~preparo_cena}\rangle$ sí es argumento.

Condición	**Exige**	**Contraejemplo en** (Π_n, Δ_n)	**Por qué falla**
(1) Derivación	$A \cup \Pi$ deriva h	Q para `en_oficina`	no deriva `trabajando`
(2) Consistencia	$A \cup \Pi$ no contradictorio	R para `en_casa`	deriva `en_casa` y `~en_casa`
(3) Minimalidad	ningún $A' \subset A$ cumple (1) y (2)	S para `~preparo_cena`	$B \subset S$ ya alcanza

EXTRAS

- **Observaciones** sobre la definición:[31]
  - el conjunto incluye **solo reglas rebatibles** (instanciadas); los hechos y reglas estrictas que se usan quedan en Π;
  - la conclusión es **única** y está instanciada;

[29] Cátedra, Episodio X, parte 2, «Argumento».
[30] Cátedra, Episodio X, parte 2, «Algunos detalles sobre las condiciones de los argumentos».
[31] Cátedra, Episodio X, parte 2, «Argumento».

- se dice que $\langle A, h\rangle$ **sustenta** h;
- para una misma conclusión puede haber **varios argumentos**, cada uno una razón diferente;
- el argumento para un **hecho** es el **conjunto vacío**.

- **Notación gráfica**: un argumento se dibuja como un **triángulo**, con la conclusión en el vértice superior y el nombre del argumento adentro. Un sub-argumento es un triángulo dentro de otro.[32]

---

b)

Consigna

Considerando el programa $\mathcal{P}_2$ del ejercicio anterior, muestre los argumentos posibles para cada uno de los siguientes literales: vuela(tina), ~vuela(tina), asustada(tina), anida_en_ramas_altas(tina).

Programa $\mathcal{P}_2$, con los nombres de reglas del ejercicio 2:

$$\Pi_2 = \left\{ \begin{array}{l} asustada(tina) \\ gallina(tina) \\ gallina(peluche) \\ \sim vuela(peluche) \\ ave(X) \leftarrow gallina(X) \end{array} \right\} \qquad \Delta_2 = \left\{ \begin{array}{l} vuela(X) \prec ave(X) \\ \sim vuela(X) \prec gallina(X) \\ vuela(X) \prec gallina(X), asustada(X) \\ anida_en_ramas_altas(X) \prec vuela(X) \end{array} \right\}$$

E1: `ave(X)` $\leftarrow$ `gallina(X)`
R1: `vuela(X)` $\prec$ `ave(X)`
R2: `~vuela(X)` $\prec$ `gallina(X)`
R3: `vuela(X)` $\prec$ `gallina(X), asustada(X)`
R4: `anida_en_ramas_altas(X)` $\prec$ `vuela(X)`

Conclusión	**Argumento**	**Derivación**
`vuela(tina)`	$A_1 = \{$`vuela(tina)` $\prec$ `ave(tina)`$\}$	D1
	$A_2 = \{$`vuela(tina)` $\prec$ `gallina(tina), asustada(tina)`$\}$	D2
`~vuela(tina)`	$B = \{$`~vuela(tina)` $\prec$ `gallina(tina)`$\}$	D3
`asustada(tina)`	$\emptyset$ (es un hecho)	D4
`anida_en_ramas_altas(tina)`	$C_1 = \{$`anida_en_ramas_altas(tina)` $\prec$ `vuela(tina)`, `vuela(tina)` $\prec$ `ave(tina)`$\}$	D1 + R4
	$C_2 = \{$`anida_en_ramas_altas(tina)` $\prec$ `vuela(tina)`, `vuela(tina)` $\prec$ `gallina(tina), asustada(tina)`$\}$	D2 + R4

Por qué cumplen las tres condiciones:

- **(1)**: cada uno contiene exactamente las reglas rebatibles de la derivación indicada (D1–D4 del ejercicio 2; para C_1 y C_2, esa derivación seguida de `anida_en_ramas_altas(tina)` por R4).
- **(2)**: con Π_2, ninguno deriva un literal y su complemento. Con Π_2 solo se deriva un literal negado, `~vuela(peluche)`, y estos argumentos no derivan `vuela(peluche)`. Además, ninguno junta R2 con R1 o R3 para tina.
- **(3)**: sacando cualquier regla ya no se deriva la conclusión.

Conjuntos que **no** son argumentos (ejemplos):

- $A_1 \cup A_2$ para `vuela(tina)`: falla (3), porque $A_1 \subset A_1 \cup A_2$ ya alcanza.
- $C_1 \cup B$ para `anida_en_ramas_altas(tina)`: falla (2), porque deriva `vuela(tina)` y `~vuela(tina)` (y también (3)).

[32] Cátedra, Episodio X (tercera parte), «Notación gráfica de argumentos».

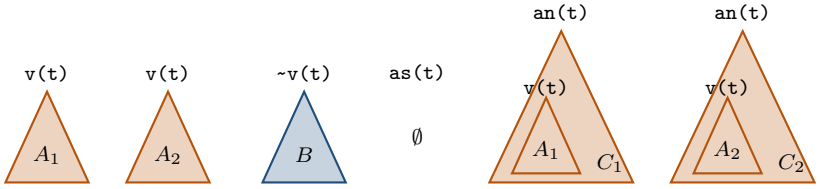


Figura 6: Los argumentos de b en notación de triángulos (`v(t)` = `vuela(tina)`, `as(t)` = `asustada(tina)`, `an(t)` = `anida_en_ramas_altas(tina)`). A_1 y A_2 son sub-argumentos de C_1 y C_2.

EXTRAS

- Un argumento $\langle S, q\rangle$ es **sub-argumento** de $\langle X, h\rangle$ si $S \subseteq X$. Por eso $\langle A_1, \texttt{vuela(tina)}\rangle$ es sub-argumento de $\langle C_1, \texttt{anida_en_ramas_altas(tina)}\rangle$ (y A_2 de C_2).[33]

c)

Consigna

¿Qué relación existe entre las nociones de argumento y derivación?

- Todo argumento **necesita una derivación**: es la condición (1), una derivación rebatible de h a partir de $A \cup \Pi$.[34]
- El conjunto A son las **reglas rebatibles instanciadas que usa esa derivación**. En el ejemplo de la cátedra, N tiene las dos reglas rebatibles (R2 y R5) de la derivación de `~comprar_acciones(acme)`; la regla estricta E3 y el hecho que usa la derivación quedan en Π.[35]
- La recíproca no vale: **no toda derivación da un argumento**. Además de derivar, A tiene que ser consistente con Π (2) y mínimo (3). Ejemplo: `vuela(peluche)` (inciso d).
- Si la derivación es **estricta**, no usa reglas rebatibles y el argumento es $\emptyset$ (p. ej. D4 para `asustada(tina)`).[36]

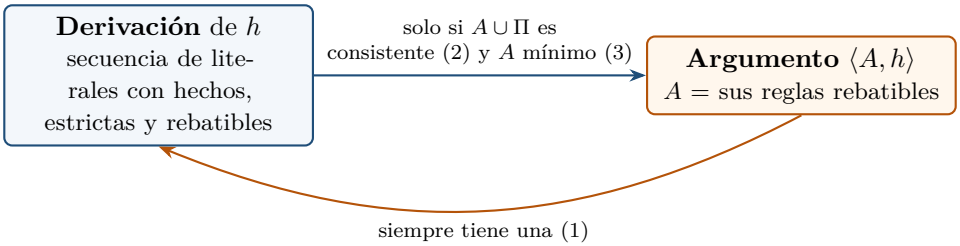


Figura 7: Todo argumento se apoya en una derivación; no toda derivación da un argumento.

[33] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos: sub-argumento».
[34] Cátedra, Episodio X, parte 2, «Argumento».
[35] Cátedra, Episodio X, parte 2, «Argumento».
[36] Cátedra, Episodio X, parte 2, «Argumento».

d)

Consigna

¿Por qué a partir del programa $\mathcal{P}_2$ existe una derivación para vuela(peluche) pero no existe un argumento para vuela(peluche)?

- **Derivación**: D5 = `gallina(peluche)`, `ave(peluche)` (E1), `vuela(peluche)` (R1). Una derivación solo pide que cada literal se obtenga de un hecho o una regla; no controla contradicciones.
- **Argumento**: el único candidato es $A = \{$`vuela(peluche)` $\prec$ `ave(peluche)`$\}$ (R3 no sirve: `asustada(peluche)` no se deriva). Pero `~vuela(peluche)` es un **hecho** de Π_2, así que $A \cup \Pi_2$ deriva `vuela(peluche)` y `~vuela(peluche)`: es contradictorio y **no cumple la condición (2)**.
- Agregar reglas no lo arregla: cualquier conjunto que derive `vuela(peluche)`, unido con Π_2, sigue derivando también el hecho `~vuela(peluche)`.

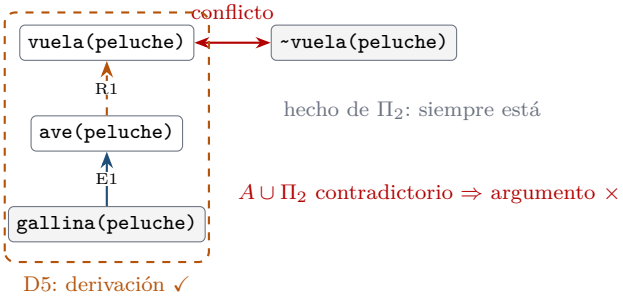


Figura 8: `vuela(peluche)` tiene derivación, pero choca con el hecho `~vuela(peluche)`.

Ejercicio 4: Ataques y derrotas

a)

Consigna

Defina formalmente las nociones de contra-argumento y derrotador (distinguiendo entre derrotador propio y derrotador por bloqueo).

Definiciones, dado un PLR (Π, Δ):

- **Desacuerdo**: $\langle X, f\rangle$ y $\langle Y, g\rangle$ están en desacuerdo (o en conflicto) si $\Pi \cup \{f, g\}$ es contradictorio.[37]
- **Contra-argumento (ataque)**: $\langle X, p\rangle$ ataca (contra-argumenta) a $\langle Y, h\rangle$ si existe un sub-argumento $\langle S, q\rangle$ de $\langle Y, h\rangle$ tal que $\langle X, p\rangle$ y $\langle S, q\rangle$ están en desacuerdo.[38]
  - $\langle S, q\rangle$ se llama **sub-argumento de desacuerdo**; puede ser el propio $\langle Y, h\rangle$.
  - q es el **punto de ataque**.
- **Derrotador**: $\langle X, p\rangle$ derrota a $\langle Y, h\rangle$ si lo ataca con sub-argumento de desacuerdo $\langle S, q\rangle$ y se cumple una de estas condiciones:[39]

  (a) $\langle X, p\rangle$ es **preferido** a $\langle S, q\rangle$: es un **derrotador propio**;

  (b) $\langle X, p\rangle$ **no es comparable** con $\langle S, q\rangle$: es un **derrotador por bloqueo**.

  La derrota combina ataque y preferencia, y la preferencia se evalúa entre el atacante y el **sub-argumento de desacuerdo** (no el argumento atacado completo).

$\langle X, p\rangle$ **ataca a** $\langle Y, h\rangle$ **en** $\langle S, q\rangle$**, y...**	**¿Derrota?**	**Tipo**
$\langle X, p\rangle$ es preferido a $\langle S, q\rangle$	sí	derrotador propio
$\langle X, p\rangle$ y $\langle S, q\rangle$ no son comparables	sí	derrotador por bloqueo
$\langle S, q\rangle$ es preferido a $\langle X, p\rangle$	no	(solo ataque)

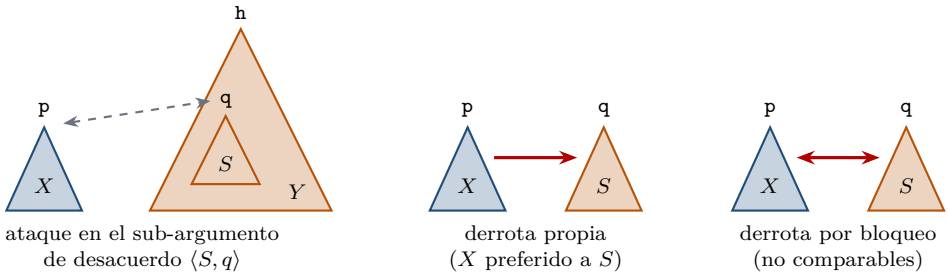


Figura 9: Ataque (flecha gris punteada), derrota propia (flecha simple) y derrota por bloqueo (flecha doble), como en la notación de la cátedra.

EXTRAS

- Todo ataque a un sub-argumento es también un ataque al súper-argumento, y si R derrota a un sub-argumento X de N, también derrota a N.[40]
- Si R derrota a N, R **defiende indirectamente** todo lo que N ataca; y un D que ataque a R defiende indirectamente a N (y ataca indirectamente lo que N ataca).[41]

[37] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos: desacuerdo».
[38] Cátedra, Episodio X (tercera parte), «Ataque (contra-argumento)».
[39] Cátedra, Episodio X (tercera parte), «Derrota propia y derrota por bloqueo».
[40] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos».
[41] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos».

b)

Consigna

A partir de los argumentos formados en el ejercicio 3.b, indique los ataques que aparecen entre los argumentos para vuela(tina) y los argumentos para ~vuela(tina).

Argumentos del ejercicio 3.b (nombres de reglas del ejercicio 2):

- $\langle A_1, \texttt{vuela(tina)}\rangle$, con $A_1 = \{\texttt{vuela(tina)} \prec \texttt{ave(tina)}\}$ (R1);
- $\langle A_2, \texttt{vuela(tina)}\rangle$, con $A_2 = \{\texttt{vuela(tina)} \prec \texttt{gallina(tina), asustada(tina)}\}$ (R3);
- $\langle B, \texttt{~vuela(tina)}\rangle$, con $B = \{\texttt{~vuela(tina)} \prec \texttt{gallina(tina)}\}$ (R2).

Ataques:

- A_1 y B **se atacan mutuamente**: $\Pi_2 \cup \{\texttt{vuela(tina)}, \texttt{~vuela(tina)}\}$ es contradictorio.
- A_2 y B **se atacan mutuamente**, por la misma razón.
- Cada argumento tiene una sola regla, así que su único sub-argumento es él mismo: el sub-argumento de desacuerdo es el **argumento completo** y el punto de ataque es la **conclusión**.
- A_1 y A_2 **no se atacan**: tienen la misma conclusión, no están en desacuerdo.

Atacante	Atacado	Sub-argumento de desacuerdo	Punto de ataque
$\langle A_1, \texttt{vuela(tina)}\rangle$	$\langle B, \texttt{~vuela(tina)}\rangle$	$\langle B, \texttt{~vuela(tina)}\rangle$	`~vuela(tina)`
$\langle B, \texttt{~vuela(tina)}\rangle$	$\langle A_1, \texttt{vuela(tina)}\rangle$	$\langle A_1, \texttt{vuela(tina)}\rangle$	`vuela(tina)`
$\langle A_2, \texttt{vuela(tina)}\rangle$	$\langle B, \texttt{~vuela(tina)}\rangle$	$\langle B, \texttt{~vuela(tina)}\rangle$	`~vuela(tina)`
$\langle B, \texttt{~vuela(tina)}\rangle$	$\langle A_2, \texttt{vuela(tina)}\rangle$	$\langle A_2, \texttt{vuela(tina)}\rangle$	`vuela(tina)`

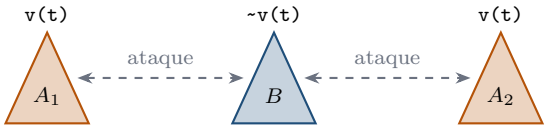


Figura 10: Ataques entre los argumentos para `vuela(tina)` (`v(t)`) y `~vuela(tina)`: B ataca y es atacado por A_1 y por A_2.

EXTRAS

- B también ataca a $\langle C_1, \texttt{anida_en_ramas_altas(tina)}\rangle$ y a $\langle C_2, \texttt{anida_en_ramas_altas(tina)}\rangle$ del ejercicio 3.b, con sub-argumentos de desacuerdo A_1 y A_2 (punto de ataque `vuela(tina)`): todo ataque a un sub-argumento es un ataque al súper-argumento. En cambio, C_1 y C_2 no atacan a B: $\Pi_2 \cup \{\texttt{anida_en_ramas_altas(tina)}, \texttt{~vuela(tina)}\}$ no es contradictorio.[42]

c)

Consigna

Defina un orden de preferencia para el conjunto de reglas rebatibles Δ_2 y, a partir de las relaciones de ataque presentadas en el inciso anterior, indique qué derrotas se producen. Para cada una, indique si se trata de una derrota propia o por bloqueo.

[42] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos».

Orden elegido. Se prioriza la regla más específica (la excepción sobre la regla general):

R2 > R1 (las gallinas no vuelan, aunque sean aves) R3 > R2 (las gallinas asustadas vuelan)

Por transitividad, también R3 > R1. R4 queda sin relacionar (no aparece en los argumentos que se comparan).

Criterio de preferencia (prioridad entre reglas). N es preferido a A si:[43]

1. existe $R_1 \in N$ y $R_2 \in A$ con $R_1 > R_2$ (N tiene al menos una regla mejor que una de A), y
2. no existe $R_1 \in N$ y $R_2 \in A$ con $R_1 < R_2$ (A no tiene ninguna regla mejor que una de N).

Comparaciones.

- B vs. A_1: R2 > R1 y A_1 no tiene reglas mejores ⇒ B **es preferido a** A_1.
- A_2 vs. B: R3 > R2 y B no tiene reglas mejores ⇒ A_2 **es preferido a** B.

Derrotas.

Ataque	Preferencia	¿Derrota?	Tipo
B ataca a A_1	B preferido a A_1	**sí**	**propia**
A_1 ataca a B	B preferido a A_1	no	—
A_2 ataca a B	A_2 preferido a B	**sí**	**propia**
B ataca a A_2	A_2 preferido a B	no	—

- $\langle B, \texttt{\sim vuela(tina)} \rangle$ es **derrotador propio** de $\langle A_1, \texttt{vuela(tina)} \rangle$.
- $\langle A_2, \texttt{vuela(tina)} \rangle$ es **derrotador propio** de $\langle B, \texttt{\sim vuela(tina)} \rangle$.
- Con este orden **no hay derrotas por bloqueo**: todos los pares que se atacan son comparables.

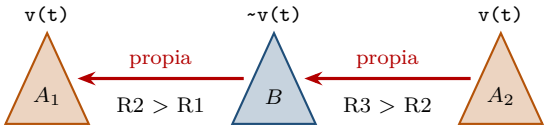
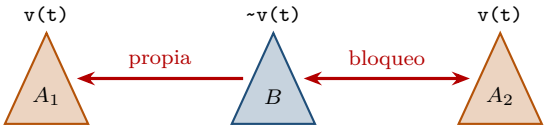


Figura 11: Derrotas con el orden R3 > R2 > R1: B derrota a A_1 y A_2 derrota a B, ambas derrotas propias.

EXTRAS

- **Otro orden posible**: solo R2 > R1 (sin comparar R3 con R2). Entonces A_2 y B no son comparables y **se derrotan mutuamente por bloqueo**; B sigue siendo derrotador propio de A_1. Es el mismo caso que $\langle A, g \rangle$ y $\langle C, \sim g \rangle$ en el ejemplo de la cátedra.[44]



- Con el orden elegido, A_2 derrota a B, que derrota a A_1: A_2 **defiende indirectamente** a A_1.[45]

[43] Cátedra, Episodio X (tercera parte), «Preferencia entre argumentos».
[44] Cátedra, Episodio X (tercera parte), «Derrota propia y derrota por bloqueo».
[45] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos».

Ejercicio 5

Consigna

Considere el programa $\mathcal{P}_5 = (\Pi_5, \Delta_5)$:

$$\Pi_5 = \left\{ \begin{array}{l} viento \leftarrow viento_caluroso \\ calor \\ caluroso \\ viento_caluroso \\ ahorrar_energia \end{array} \right\} \qquad \Delta_5 = \left\{ \begin{array}{l} aire_acond \prec calor \\ aire_acond \prec sofocante \\ \sim aire_acond \prec abrir_ventana \\ \sim aire_acond \prec ahorrar_energia \\ abrir_ventana \prec calor, viento \\ \sim abrir_ventana \prec viento_caluroso \end{array} \right\}$$

En lo que sigue, las reglas se nombran así:

R1:	`aire_acond` $\prec$ `calor`	R4:	`~aire_acond` $\prec$ `ahorrar_energia`
R2:	`aire_acond` $\prec$ `sofocante`	R5:	`abrir_ventana` $\prec$ `calor, viento`
R3:	`~aire_acond` $\prec$ `abrir_ventana`	R6:	`~abrir_ventana` $\prec$ `viento_caluroso`

`sofocante` no se deriva

R2 nunca se puede usar: `sofocante` no es un hecho ni cabeza de ninguna regla. En Π_5 está el hecho `caluroso`, que ninguna regla usa; probablemente R2 debía ser `aire_acond` $\prec$ `caluroso`. Se resuelve el programa tal como está.

a)

Consigna

Brinde dos argumentos para no encender el aire acondicionado (~aire_acond) y muestre todos los contra-argumentos que los atacan.

Argumentos para `~aire_acond`.

- $\langle A_1, \texttt{~aire_acond}\rangle$, con $A_1 = \{\texttt{~aire_acond} \prec \texttt{abrir_ventana}, \texttt{abrir_ventana} \prec \texttt{calor, viento}\}$ (R3, R5). `calor` es hecho y `viento` sale de la regla estricta con `viento_caluroso`.
  - Tiene el sub-argumento $\langle D, \texttt{abrir_ventana}\rangle$, con $D = \{\texttt{abrir_ventana} \prec \texttt{calor, viento}\}$ (R5).
- $\langle A_2, \texttt{~aire_acond}\rangle$, con $A_2 = \{\texttt{~aire_acond} \prec \texttt{ahorrar_energia}\}$ (R4).

Contra-argumentos. Un contra-argumento tiene que estar en desacuerdo con algún sub-argumento: concluir `aire_acond` (contra A_1 o A_2) o `~abrir_ventana` (contra D).[46]

- $\langle B, \texttt{aire_acond}\rangle$, con $B = \{\texttt{aire_acond} \prec \texttt{calor}\}$ (R1). Es el único argumento para `aire_acond` (R2 no se puede usar).
- $\langle C, \texttt{~abrir_ventana}\rangle$, con $C = \{\texttt{~abrir_ventana} \prec \texttt{viento_caluroso}\}$ (R6).

Contra-argumento	Ataca a	Sub-argumento de desacuerdo	Punto de ataque
$\langle B, \texttt{aire_acond}\rangle$	$\langle A_1, \texttt{~aire_acond}\rangle$	$\langle A_1, \texttt{~aire_acond}\rangle$	`~aire_acond`
$\langle C, \texttt{~abrir_ventana}\rangle$	$\langle A_1, \texttt{~aire_acond}\rangle$	$\langle D, \texttt{abrir_ventana}\rangle$	`abrir_ventana`
$\langle B, \texttt{aire_acond}\rangle$	$\langle A_2, \texttt{~aire_acond}\rangle$	$\langle A_2, \texttt{~aire_acond}\rangle$	`~aire_acond`

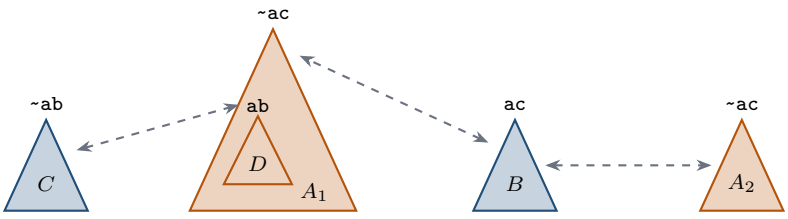


Figura 12: Ataques a los argumentos para `~aire_acond` (`ac` = `aire_acond`, `ab` = `abrir_ventana`). C ataca a A_1 en su sub-argumento D.

b)

Consigna

Compare los argumentos obtenidos a partir del criterio de preferencia entre reglas visto en clase. En este caso, considere el siguiente orden:

$$(\texttt{aire_acond} \prec \texttt{sofocante}) > (\texttt{~aire_acond} \prec \texttt{ahorrar_energia})$$
$$(\texttt{~aire_acond} \prec \texttt{ahorrar_energia}) > (\texttt{aire_acond} \prec \texttt{calor})$$
$$(\texttt{~abrir_ventana} \prec \texttt{viento_caluroso}) > (\texttt{abrir_ventana} \prec \texttt{calor, viento})$$

Determine si los contra-argumentos obtenidos en el inciso anterior son derrotadores. En caso afirmativo, indique si son derrotadores propios o por bloqueo.

Con los nombres de reglas: **R2** > **R4**, **R4** > **R1**, **R6** > **R5** (y R2 > R1 por transitividad). R3 no está relacionada con ninguna regla.

[46] Cátedra, Episodio X (tercera parte), «Ataque (contra-argumento)».

La preferencia se evalúa entre el atacante y el **sub-argumento de desacuerdo**; N es preferido a A si tiene una regla mejor que una de A y A no tiene ninguna mejor que una de N.[47]

Atacante	Sub-arg. de desacuerdo	Reglas	Comparación	Resultado
B	A_1	{R1} vs. {R3, R5}	sin relación entre R1 y R3, R5	**bloqueo**
C	D (de A_1)	{R6} vs. {R5}	R6 > R5: C preferido	**propio**
B	A_2	{R1} vs. {R4}	R4 > R1: A_2 preferido	no derrota

- $\langle B, \texttt{aire_acond}\rangle$ es **derrotador por bloqueo** de $\langle A_1, \texttt{\~aire_acond}\rangle$: no son comparables.
- $\langle C, \texttt{\~abrir_ventana}\rangle$ es **derrotador propio** de $\langle A_1, \texttt{\~aire_acond}\rangle$: es preferido a D, el sub-argumento atacado.
- $\langle B, \texttt{aire_acond}\rangle$ ataca a $\langle A_2, \texttt{\~aire_acond}\rangle$ pero **no es derrotador**: A_2 es preferido a B.
- La regla R2 aparece en el orden pero no influye: no está en ningún argumento.

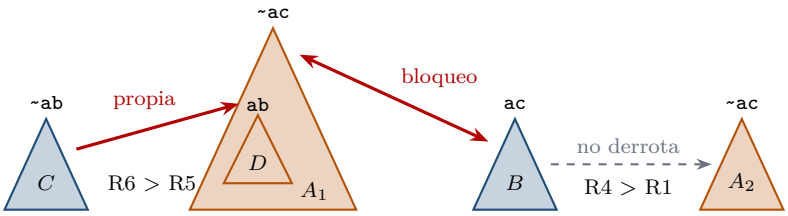


Figura 13: Derrotas sobre los argumentos para `~aire_acond`: C derrota a A_1 (propia, en D), B y A_1 se bloquean, y B no derrota a A_2.

EXTRAS

- **Ataques en sentido contrario**, que se usan en los ejercicios 6 y 8:[48]
  - A_1 ataca a B y, como no son comparables, también es su **derrotador por bloqueo**.
  - A_2 ataca a B y es su **derrotador propio** (R4 > R1).
  - D ataca a C pero no lo derrota (C es preferido). A_1 no ataca a C: `~aire_acond` y `~abrir_ventana` junto con Π_5 no forman un conjunto contradictorio; el ataque es solo de su sub-argumento D.

[47] Cátedra, Episodio X (tercera parte), «Derrota propia y derrota por bloqueo»; «Preferencia entre argumentos».
[48] Cátedra, Episodio X (tercera parte), «Derrota propia y derrota por bloqueo».

Ejercicio 6: Líneas de argumentación

a)

Consigna

¿Qué es una línea de argumentación? ¿Qué condiciones deben cumplirse para que una línea de argumentación sea aceptable?

Línea de argumentación. Una línea de argumentación para $\langle A_0, h_0 \rangle$ es una **secuencia finita de argumentos** $[A_0, A_1, A_2, \dots]$ que comienza con A_0 y en la que, a partir del segundo, **cada argumento es un derrotador del anterior**.[49]

- A_1 derrota a A_0, pero A_2 derrota a A_1, así que **defiende** a A_0; y así sucesivamente.
- **Argumentos de soporte** para A_0: los que lo defienden, $\{A_0, A_2, A_4, \dots\}$ (posiciones pares).
- **Argumentos de interferencia** para A_0: los que lo derrotan directa o indirectamente, $\{A_1, A_3, \dots\}$ (posiciones impares).

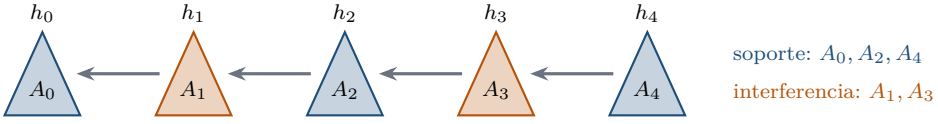
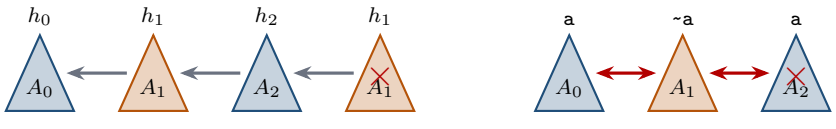


Figura 14: Una línea de argumentación para A_0: cada argumento derrota al anterior (la flecha va del derrotador al derrotado).

Línea aceptable. Una línea de argumentación para A_0 es **aceptable** si:[50]

1. Ningún argumento A_k **aparece más de una vez** en la línea, ni es un **sub-argumento de desacuerdo** de otro A_i que aparece antes ($i < k$).
2. El conjunto de argumentos de **soporte** para A_0 **no es contradictorio**, y tampoco el de **interferencia**. Un conjunto de argumentos $\{A_1, \dots, A_N\}$ es contradictorio si $\Pi \cup A_1 \cup \dots \cup A_N$ deriva un par de literales complementarios p y $\sim p$.
3. Si A_k es un **derrotador por bloqueo** de A_{k-1}, entonces A_{k+1} debe ser un **derrotador propio** de A_k: no puede haber dos derrotadores por bloqueo consecutivos.

Condición	Exige	Evita
(1) Sin repetición	ningún argumento repetido ni sub-argumento de desacuerdo de uno anterior	líneas con ciclos (infinitas)
(2) Consistencia	soporte e interferencia no contradictorios por separado	que un bando use argumentos que se contradicen entre sí
(3) Sin doble bloqueo	después de un bloqueo, una derrota propia	dos derrotadores por bloqueo seguidos



(1) A_1 repetido (3) dos bloqueos seguidos

Figura 15: Dos líneas no aceptables: el argumento marcado con × no puede agregarse.

[49] Cátedra, Episodio X (tercera parte), «Línea de argumentación para un argumento».
[50] Cátedra, Episodio X (tercera parte), «Línea de argumentación aceptable». Lo que evita cada condición: «(1) evitando líneas con ciclos (infinitos)», «(2) evitando contradicciones», «(3) evitando doble bloqueo».

EXTRAS

- **Por qué hace falta la condición (1)**: en el PLR Merval, B (`buen_precio(skynet)`) y M (`~buen_precio(skynet)`) se bloquean mutuamente, y sin restricciones la línea $[B, M, B, M, \dots]$ sería infinita.[51]
- **Por qué alcanza con no agregar el segundo bloqueo** (3): el bloqueo es simétrico; si A_1 bloquea a A_0, A_0 también bloquea a A_1, así que A_1 ya está bloqueado, se agregue o no A_2.[52]
- En un árbol de dialéctica, **todo camino de la raíz a una hoja es una línea de argumentación**.[53]

b)

Consigna

Considerando el programa $\mathcal{P}_5$ del ejercicio anterior, determine todas las líneas de argumentación aceptables para los argumentos cuya conclusión sea el literal aire_acond o el literal ~aire_acond.

Programa $\mathcal{P}_5$, con los nombres de reglas del ejercicio 5 y el orden R2 > R4, R4 > R1, R6 > R5:

$$\Pi_5 = \left\{ \begin{array}{l} viento \leftarrow viento_caluroso \\ calor \\ caluroso \\ viento_caluroso \\ ahorrar_energia \end{array} \right\} \qquad \Delta_5 = \left\{ \begin{array}{l} aire_acond \prec calor \\ aire_acond \prec sofocante \\ \sim aire_acond \prec abrir_ventana \\ \sim aire_acond \prec ahorrar_energia \\ abrir_ventana \prec calor, viento \\ \sim abrir_ventana \prec viento_caluroso \end{array} \right\}$$

R1: `aire_acond` $\prec$ `calor`
R2: `aire_acond` $\prec$ `sofocante`
R3: `~aire_acond` $\prec$ `abrir_ventana`
R4: `~aire_acond` $\prec$ `ahorrar_energia`
R5: `abrir_ventana` $\prec$ `calor, viento`
R6: `~abrir_ventana` $\prec$ `viento_caluroso`

Argumentos (del ejercicio 5):

- $\langle A_1, \texttt{\~aire_acond} \rangle$, $A_1 = \{\text{R3, R5}\}$, con sub-argumento $\langle D, \texttt{abrir_ventana} \rangle$, $D = \{\text{R5}\}$;
- $\langle A_2, \texttt{\~aire_acond} \rangle$, $A_2 = \{\text{R4}\}$;
- $\langle B, \texttt{aire_acond} \rangle$, $B = \{\text{R1}\}$;
- $\langle C, \texttt{\~abrir_ventana} \rangle$, $C = \{\text{R6}\}$.

Derrotas (ejercicio 5.b y sus EXTRAS):

Argumento	**Sus derrotadores**	**Tipo**
A_1	B	bloqueo
	C (en el sub-argumento D)	propio
A_2	—	
B	A_1	bloqueo
	A_2	propio
C	— (D lo ataca, pero C es preferido)	

[51] Cátedra, Episodio X (tercera parte), «¿ Argumentación infinita ?»; «Línea de argumentación aceptable».
[52] Cátedra, Episodio X (tercera parte), «(3) evitando doble bloqueo».
[53] Cátedra, Episodio X (tercera parte), «Líneas de argumentación en un árbol de dialéctica».

Líneas aceptables. Se construyen agregando derrotadores mientras se cumplan las tres condiciones:

Raíz	Línea	Soporte	Interferencia	Por qué termina
A_1	$\Lambda_1 = [A_1, C]$	$\{A_1\}$	$\{C\}$	C no tiene derrotadores
	$\Lambda_2 = [A_1, B, A_2]$	$\{A_1, A_2\}$	$\{B\}$	A_2 no tiene derrotadores
A_2	$\Lambda_3 = [A_2]$	$\{A_2\}$	$\emptyset$	A_2 no tiene derrotadores
B	$\Lambda_4 = [B, A_2]$	$\{B\}$	$\{A_2\}$	A_2 no tiene derrotadores
	$\Lambda_5 = [B, A_1, C]$	$\{B, C\}$	$\{A_1\}$	C no tiene derrotadores

Por qué son aceptables:

- **(1)**: en ninguna se repite un argumento, y ninguno es sub-argumento de desacuerdo de uno anterior.
- **(2)**: ningún conjunto de soporte o de interferencia es contradictorio. Los únicos con dos argumentos son $\{A_1, A_2\}$ (ambos concluyen `~aire_acond`) y $\{B, C\}$ (`aire_acond` y `~abrir_ventana`); junto con Π_5 no derivan ningún par p, $\sim p$.
- **(3)**: en Λ_2 y Λ_5, al bloqueo del segundo argumento le sigue una **derrota propia** (A_2 sobre B; C sobre A_1 en D).

Líneas descartadas:

- $[A_1, B, A_1]$ y $[B, A_1, B]$: violan **(1)** (argumento repetido) y **(3)** (dos bloqueos seguidos).

Líneas más cortas

Los prefijos de una línea aceptable también son líneas aceptables (p. ej. $[A_1]$, $[A_1, B]$, $[B]$, $[B, A_1]$). Arriba se muestran solo las **maximales**, que no se pueden extender: son las que aparecen como caminos de la raíz a una hoja en los árboles de dialéctica.

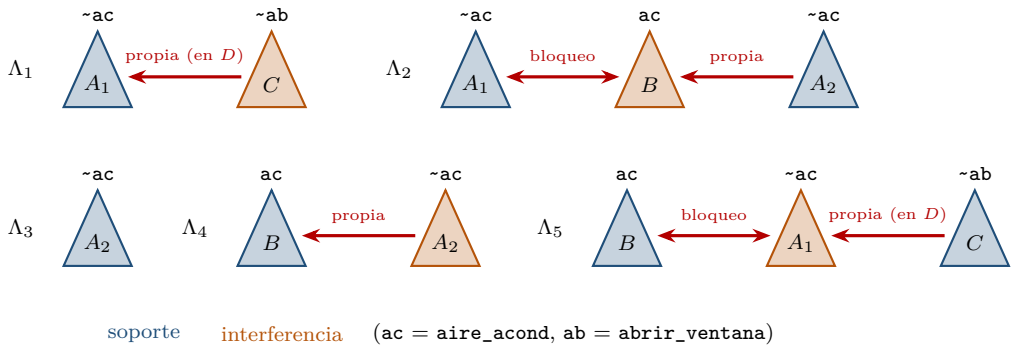


Figura 16: Las cinco líneas de argumentación aceptables maximales para los argumentos de `aire_acond` y `~aire_acond`.

Ejercicio 7

Consigna

Indique si las siguientes líneas de argumentación son aceptables. Justifique adecuadamente en cada caso apelando a las condiciones enunciadas en el ejercicio 6.a.
(Siguen cuatro líneas, (a)–(d), dadas como figura; abajo se redibujan una por una.)

Cómo se lee la figura

- Cada triángulo es un argumento, con su conclusión arriba; un triángulo interior es un sub-argumento. Dos triángulos con el mismo nombre son el mismo argumento.
- Cada flecha es una derrota y apunta al **sub-argumento de desacuerdo** del derrotado (al vértice, si ataca la conclusión).
- La figura **no indica** si las derrotas son propias o por bloqueo, así que la condición (3) solo puede evaluarse de forma condicional.

Se aplican las tres condiciones del ejercicio 6.a. La que decide en los cuatro casos es la (1): el argumento que se agrega no puede estar **repetido** ni ser el **sub-argumento de desacuerdo** de uno anterior.[54]

(a) $[A_0, X, A_1, X]$: no es aceptable.

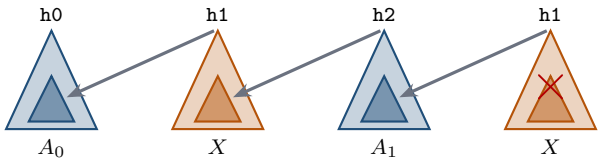


Figura 17: Línea (a): X aparece dos veces.

- X aparece **dos veces** (posiciones 2 y 4): viola la condición **(1)**.

(b) $[A_0, A_1, X]$, con A_1 atacando en $\langle X, h_4\rangle$: no es aceptable.

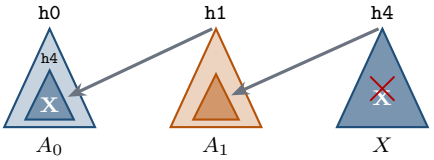


Figura 18: Línea (b): X es el sub-argumento de desacuerdo de A_0.

- A_1 derrota a A_0 atacando su sub-argumento $\langle X, h_4\rangle$: ese es el **sub-argumento de desacuerdo** de A_0.
- X, que ya fue derrotado por A_1, vuelve a entrar para derrotar a A_1: es sub-argumento de desacuerdo de un argumento anterior (A_0) y viola la condición **(1)**. Sería una argumentación circular: X se defendería a sí mismo.

(c) $[A_0, A_1, X]$, con A_1 atacando la conclusión h_0: es aceptable (salvo doble bloqueo).

[54] Cátedra, Episodio X (tercera parte), «Línea de argumentación aceptable»; «(1) evitando líneas con ciclos (infinitos)». Las líneas (a) y (b) son las dos de esta última diapositiva.

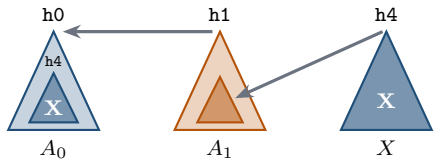


Figura 19: Línea (c): X es sub-argumento de A_0, pero no el de desacuerdo.

- **(1) se cumple**: no hay repetidos, y A_1 ataca a A_0 en su conclusión, así que el sub-argumento de desacuerdo es A_0 completo. X es sub-argumento de A_0, pero **no el de desacuerdo**.
- **(2) se cumple**:
  - Soporte $\{A_0, X\}$: como $X \subseteq A_0$, $\Pi \cup A_0 \cup X = \Pi \cup A_0$, que no es contradictorio porque A_0 es un argumento.[55]
  - Interferencia $\{A_1\}$: un solo argumento, tampoco es contradictorio.
- **(3)** depende del tipo de derrota: se viola solo si A_1 bloquea a A_0 **y** X bloquea a A_1.

(d) $[A_0, A_1, X]$, con A_1 atacando en $\langle S, h_5\rangle$: es aceptable (salvo doble bloqueo).

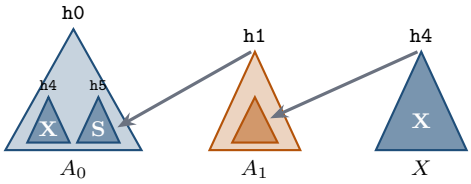


Figura 20: Línea (d): A_1 ataca a A_0 en S; X es otro sub-argumento de A_0.

- **(1) se cumple**: el sub-argumento de desacuerdo de A_0 es $\langle S, h_5\rangle$ (S es el nombre que se le da acá al sub-argumento para h_5), no X.
- **(2)** y **(3)**: igual que en (c), porque el soporte vuelve a ser $\{A_0, X\}$ con $X \subseteq A_0$ y los tipos de derrota tampoco se indican.

Resumen.

Línea	(1)	(2)	(3)	¿Aceptable?
(a)	no: X repetido	—	—	**no**
(b)	no: X es sub-arg. de desacuerdo de A_0	—	—	**no**
(c)	sí: el desacuerdo es en h_0	sí	si no hay doble bloqueo	**sí**
(d)	sí: el desacuerdo es en h_5	sí	si no hay doble bloqueo	**sí**

EXTRAS

- **Ejemplo de la cátedra del caso (b)**, en el programa de `surf`: $\langle L, \texttt{~nice}\rangle$ derrota por bloqueo a $\langle A, \texttt{surf}\rangle$ en el sub-argumento de desacuerdo $\langle N, \texttt{nice}\rangle$. Aunque N derrota por bloqueo a L, no puede agregarse después de L, porque es el sub-argumento de desacuerdo de A.[56]

[55] Cátedra, Episodio X, parte 2, «Argumento».
[56] Cátedra, Episodio X (tercera parte), «Verificando que la línea de argumentación sea aceptable».

Ejercicio 8: Análisis dialéctico y respuesta a consultas

a)

Consigna

¿Qué es un árbol dialéctico?

Árbol de dialéctica para $\langle A_0, h_0 \rangle$.

- La **raíz** se etiqueta con $\langle A_0, h_0 \rangle$.[57]
- Para un nodo $\langle A_N, h_N \rangle$, con camino desde la raíz $[A_0, \ldots, A_N]$: cada **derrotador** $\langle B_i, q_i \rangle$ de A_N tal que $[A_0, \ldots, A_N, B_i]$ es una **línea de argumentación aceptable** es un **hijo** de A_N.
- Si A_N no tiene ningún derrotador en esas condiciones, es una **hoja**.

Marcado. Para saber si la raíz resulta derrotada, DeLP marca los nodos:[58]

- **U** (*undefeated*): toda **hoja**, y todo nodo interno cuyos hijos están **todos marcados D**.
- **D** (*defeated*): todo nodo interno con **al menos un hijo marcado U**.

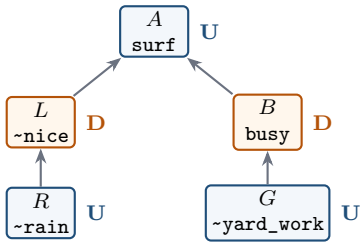


Figura 21: Árbol de dialéctica marcado de la cátedra para `surf`: las hojas R y G son U, así que L y B son D y la raíz A es U. La flecha va del derrotador al derrotado.

EXTRAS

- En ese ejemplo, L y B derrotan a A en sus sub-argumentos de desacuerdo $\langle N, \texttt{nice} \rangle$ y $\langle T, \texttt{~busy} \rangle$, y R y G son derrotadores propios de L y B en sus sub-argumentos para `rain` y `yard_work`. La respuesta para `surf` es **yes**.[59]

b)

Consigna

Si la raíz de un árbol dialéctico está marcada como U, ¿es posible asegurar que todos los argumentos que defienden a la raíz van a estar marcados U, y todos aquellos que atacan (directa o indirectamente) a la raíz van a estar marcados D? Muestre un ejemplo que justifique su respuesta.

No.

- Que la raíz sea U solo exige que **sus hijos** sean todos D, y a cada hijo D le alcanza con **un** hijo U. Los demás nodos de esos subárboles pueden tener cualquier marca.

[57] Cátedra, Episodio X (tercera parte), «Definición de Árbol de dialéctica».
[58] Cátedra, Episodio X (tercera parte), «Árbol de dialéctica marcado».
[59] Cátedra, Episodio X (tercera parte), «Marcado completo del árbol»; «Respuesta: sí, disfrutemos de un día de surf».

- Ejemplo (árbol de la cátedra): la raíz B es U, pero el argumento de soporte E_2 (nivel 2, defiende a B) está marcado **D**, y el de interferencia F (nivel 3) está marcado **U**.[60]

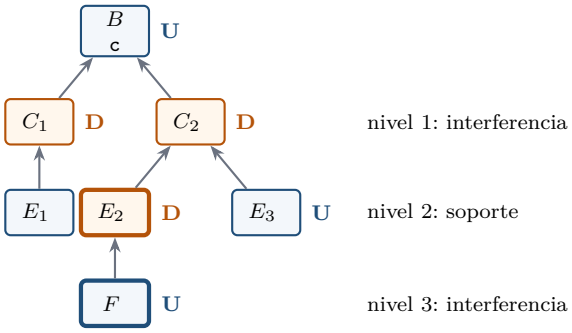


Figura 22: Raíz U con un argumento de soporte D (E_2) y uno de interferencia U (F), resaltados.

c)

Consigna

¿Es necesario analizar todas las ramas de un árbol dialéctico para conocer el marcado de los nodos? Muestre un ejemplo que justifique su respuesta.

No, no siempre. Se sigue de la regla de marcado:[61]

- Apenas un nodo tiene **un hijo U**, ya es **D**: los subárboles de sus otros hijos no cambian su marca y pueden no explorarse (**poda**).
- Para que un nodo sea **U**, en cambio, hay que comprobar que **todos** sus hijos sean D.

Ejemplo, con $\mathcal{P}_5$: el árbol para $\langle B, \texttt{aire_acond}\rangle$ tiene dos hijos, A_2 y A_1 (ejercicio 6.b). Si primero se explora A_2, que es una hoja (U), B queda marcado D sin analizar el subárbol de A_1.

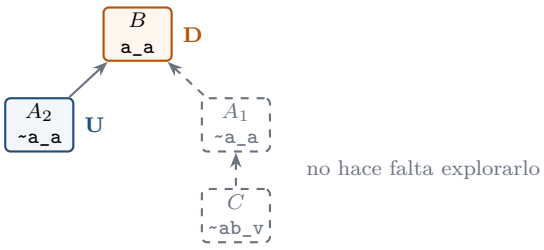


Figura 23: Poda: como A_2 es U, B es D sin importar la rama de A_1 (punteada). `a_a` = `aire_acond`, `ab_v` = `abrir_ventana`.

EXTRAS

- La cátedra pide a veces los árboles "**sin podas**", es decir, completos, aunque no hagan falta para el marcado.[62]

[60] Cátedra, Episodio X (tercera parte), «Conclusión garantizada».
[61] Cátedra, Episodio X (tercera parte), «Árbol de dialéctica marcado».
[62] Cátedra, «Resolución con DeLP» (ejemplo resuelto).

d)

Consigna

¿Cuándo se dice que un literal está garantizado? ¿Qué relación existe entre la garantía de un literal y la de su complemento?

- Un literal c está **garantizado** por un programa DeLP si existe **al menos un árbol de dialéctica** para un argumento que sustenta c cuya **raíz está marcada U**.[63]
- Por eso, ante una consulta c, DeLP analiza los argumentos para c y los argumentos para $\sim c$.
- **Relación con el complemento**:
  - c y $\sim c$ **no pueden estar garantizados a la vez**.[64]
  - Puede que **ninguno** esté garantizado (situación de indecisión).[65]

c	$\sim c$	**¿Posible?**
garantizado	no garantizado	sí
no garantizado	garantizado	sí
no garantizado	no garantizado	sí (indecisión)
garantizado	garantizado	**no**

EXTRAS

- En los ejemplos de la cátedra, un literal puede tener varios argumentos y varios árboles: alcanza con que **uno** tenga la raíz U, aunque los otros tengan la raíz D.[66]

e)

Consigna

¿Cuáles son las posibles respuestas de un intérprete DeLP?

Para una consulta c a un programa $\mathcal{P}$:[67]

- **yes**: c está garantizado por $\mathcal{P}$.
- **no**: el **complemento** de c está garantizado por $\mathcal{P}$ (complemento$(c) = \sim c$ y complemento$(\sim c) = c$).
- **undecided**: c pertenece a $\mathcal{P}$, pero ni c ni su complemento están garantizados.
- **unknown**: c **no pertenece** a $\mathcal{P}$, es decir, ni c ni su complemento aparecen en alguna regla de $\mathcal{P}$.

[63] Cátedra, Episodio X (tercera parte), «Conclusión garantizada».
[64] No está en la cátedra. García & Simari, *Defeasible Logic Programming: An Argumentative Approach*, TPLP 4(1–2), 2004 (referencia [2] del Episodio X, tercera parte).
[65] Cátedra, Episodio X (tercera parte), «Situación de indecisión».
[66] Cátedra, Episodio X (tercera parte), «Conclusión garantizada».
[67] Cátedra, Episodio X (tercera parte), «Respuesta a una consulta». Misma definición en Cátedra, «Resolución con DeLP».

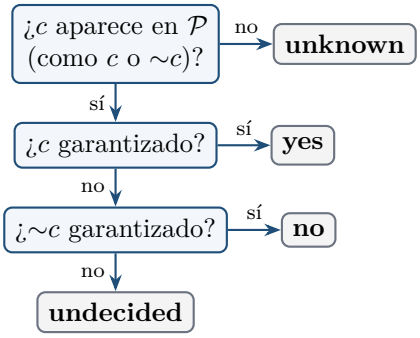


Figura 24: Respuesta del intérprete para una consulta c.

f)

Consigna

Determine qué literales están garantizados según el programa $\mathcal{P}_5$, construyendo los árboles dialécticos correspondientes.

Programa $\mathcal{P}_5$, con el orden R2 > R4, R4 > R1, R6 > R5:

$$\Pi_5 = \left\{ \begin{array}{l} \mathit{viento} \leftarrow \mathit{viento_caluroso} \\ \mathit{calor} \\ \mathit{caluroso} \\ \mathit{viento_caluroso} \\ \mathit{ahorrar_energia} \end{array} \right\} \qquad \Delta_5 = \left\{ \begin{array}{l} \mathit{aire_acond} \prec \mathit{calor} \\ \mathit{aire_acond} \prec \mathit{sofocante} \\ \sim \mathit{aire_acond} \prec \mathit{abrir_ventana} \\ \sim \mathit{aire_acond} \prec \mathit{ahorrar_energia} \\ \mathit{abrir_ventana} \prec \mathit{calor}, \mathit{viento} \\ \sim \mathit{abrir_ventana} \prec \mathit{viento_caluroso} \end{array} \right\}$$

R1: `aire_acond` $\prec$ `calor`
R2: `aire_acond` $\prec$ `sofocante`
R3: `~aire_acond` $\prec$ `abrir_ventana`
R4: `~aire_acond` $\prec$ `ahorrar_energia`
R5: `abrir_ventana` $\prec$ `calor, viento`
R6: `~abrir_ventana` $\prec$ `viento_caluroso`

Argumentos y derrotas. Son los del ejercicio 6.b (A_1, A_2, B, C). Hay que agregar los dos que faltan:

- $\langle D, \texttt{abrir_ventana} \rangle$, $D = \{\text{R5}\}$ (sub-argumento de A_1). Su único derrotador es C, **propio** (R6 > R5): `abrir_ventana` y `~abrir_ventana` están en desacuerdo.
- Los literales de Π_5 (`calor`, `caluroso`, `viento_caluroso`, `ahorrar_energia` y `viento`) tienen como argumento el **conjunto vacío**.[68] No tienen derrotadores: un argumento para el complemento sería contradictorio con Π_5, así que no sería argumento.
- `sofocante` no tiene argumento (no se deriva; ver el aviso del ejercicio 5), y por eso `aire_acond` tampoco tiene un argumento con R2.

Árboles.

[68] Cátedra, Episodio X, parte 2, «Argumento».

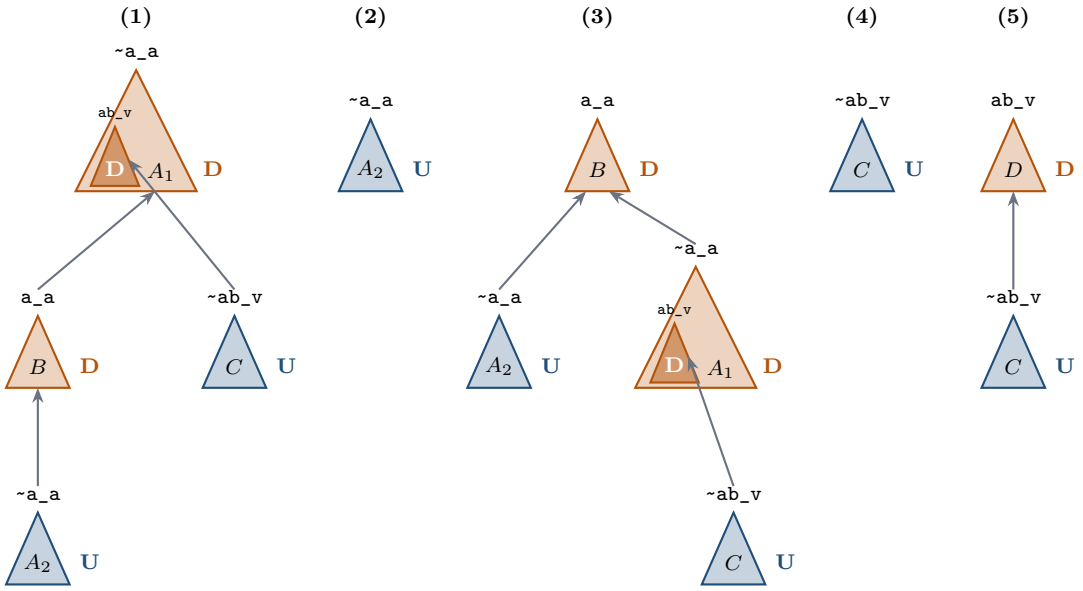


Figura 25: Árboles de dialéctica marcados de $\mathcal{P}_5$ (azul: U; ámbar: D). `a_a` = `aire_acond`, `ab_v` = `abrir_ventana`. El triángulo interior de A_1 es su sub-argumento $\langle D, \texttt{ab_v}\rangle$. La flecha va del derrotador al punto de ataque: al sub-argumento D, o a la base del triángulo si ataca la conclusión.

(1) A_1: hijos B (bloqueo) y C (propio, en D). Bajo B va A_2 (propio); A_1 no puede volver a entrar (condición 1). A_2 y C son hojas: U. B queda D y A_1 queda **D**.

(2) A_2: no tiene derrotadores. Hoja: **U**.

(3) B: hijos A_2 (propio) y A_1 (bloqueo). Bajo A_1 va C (propio, en D), sin doble bloqueo. A_2 y C son U, A_1 es D y B queda **D**.

(4) C: no tiene derrotadores (D lo ataca, pero C es preferido). Hoja: **U**.

(5) D: hijo C (propio). C es U, así que D queda **D**.

Literales garantizados.

Literal	¿Garantizado?	Por qué
`calor`, `caluroso`, `viento_caluroso`, `ahorrar_energia`, `viento`	sí	argumento $\emptyset$ sin derrotadores: árbol de un solo nodo, U
`~aire_acond`	**sí**	árbol (2) con raíz U (aunque el árbol (1) tenga raíz D)
`~abrir_ventana`	**sí**	árbol (4) con raíz U
`aire_acond`	no	su único árbol (3) tiene raíz D
`abrir_ventana`	no	su único árbol (5) tiene raíz D
`sofocante`	no	no tiene argumento

EXTRAS

- **Respuestas del intérprete** para estos literales, según las definiciones del inciso e):[69]

Consulta	Respuesta
literales de Π_5, `~aire_acond`, `~abrir_ventana`	yes
`aire_acond`, `abrir_ventana`	no (su complemento está garantizado)
`sofocante`	undecided (aparece en R2, pero ni él ni su complemento están garantizados)

[69] Cátedra, Episodio X (tercera parte), «Respuesta a una consulta».

g)

Consigna

¿Puede aparecer el mismo argumento más de una vez en el mismo árbol dialéctico? En caso afirmativo, ¿dicho argumento siempre va a quedar marcado igual?

- **Sí**: un argumento no puede repetirse dentro de una **línea** (condición 1 del ejercicio 6.a), pero sí en **líneas diferentes** del mismo árbol.[70]
- **No** siempre queda marcado igual: la marca depende del subárbol que tiene debajo, y en cada línea los derrotadores admitidos pueden ser distintos.

Ejemplo de la cátedra:[71]

- $\Pi = \{x, y, z, u\}$; preferencia $(b \prec e) > (\sim b \prec y)$.
- $A = \{a \prec d;\ d \prec x\}$, $C_2 = \{\sim a \prec \sim b, e;\ \sim b \prec y;\ e \prec u\}$, $C_3 = \{\sim d \prec \sim b;\ \sim b \prec y\}$, $X = \{b \prec e;\ e \prec u\}$, $C_4 = \{\sim e \prec z\}$.
- X aparece dos veces, las dos como argumento de soporte para A:
  - En $[A, C_2, X]$ es **U**: C_4 no puede derrotarlo ahí, porque C_2 y C_4 son de interferencia y contradictorios (e y $\sim e$), lo que viola la condición 2.
  - En $[A, C_3, X, C_4]$ es **D**: esa línea sí es aceptable, y C_4 (hoja, U) lo derrota.

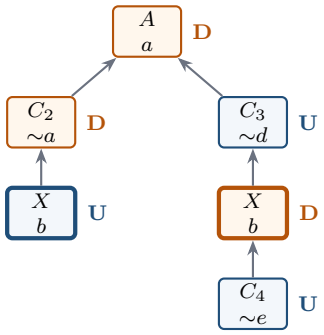


Figura 26: El mismo argumento X (resaltado) aparece dos veces en el árbol, con marcas distintas.

EXTRAS

- En otro programa de la cátedra, X aparece **tres veces** en el mismo árbol, siempre U pero en niveles distintos: dos veces como soporte y una como interferencia, porque derrota a tres argumentos diferentes.[72]
- También puede cambiar el **rol**: en un tercer ejemplo, X aparece una vez como soporte (U) y otra como interferencia (D).[73]

[70] Cátedra, Episodio X (tercera parte), «Argumentos repetidos en un árbol de dialéctica».
[71] Cátedra, Episodio X (tercera parte), «Argumentos en diferentes partes de un árbol de dialéctica».
[72] Cátedra, Episodio X (tercera parte), «Argumentos repetidos en un árbol de dialéctica».
[73] Cátedra, Episodio X (tercera parte), «Argumentos repetidos en diferentes líneas y con diferente rol y marca».

h)

Consigna

Además de para determinar la respuesta del intérprete ¿para qué más nos pueden servir los árboles dialécticos?

- Como **explicación** de la respuesta: el árbol muestra qué argumentos sustentan la conclusión, cuáles la atacan, quién derrota a quién y por qué la raíz queda U o D.[74]

EXTRAS

- La cátedra lo cuenta entre las características de DeLP: *es posible dar una explicación detallada de las respuestas*. Una de sus referencias trata el tema: García & Simari, *Defeasible Logic Programming: DeLP-servers, Contextual Queries, and Explanations for Answers* (2014).[75]

[74] Cátedra, Episodio X (tercera parte), «Respuesta a una consulta» («Los árboles de dialéctica son, además, una explicación para las respuestas»); «Respuesta: sí, disfrutemos de un día de surf».
[75] Cátedra, Episodio X (tercera parte), «Comentarios finales».